A - 栈

认识数据结构中 **栈(Stack)** 的相关用法，这道题用stack、vector、或者自己实现都可以，需要注意的是数据范围 2^{64} 超过了**long long** 的 $2^{63}-1$ 的上界， 所以至少要用 **unsigned long long**

```cpp
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
using u64 = unsigned long long;
using i128 = __int128_t;

const int maxn = 1e6 + 5;

struct Stack{
    u64 st[maxn];
    int p;

    Stack() : p(0) {}
    void push(u64 x) { st[p++] = x;}
    void pop(){p--;}
    u64 top(){ return st[p - 1];}
    u64 size() {return p;}
};

void solve(){
    Stack st;
    int n;
    u64 x;
    string op;
    cin >> n;
    while(n--){
        cin >> op;
        if(op == "push"){
            cin >> x;
            st.push(x);
        } else if(op == "pop"){
            if(st.size() == 0) cout << "Empty\n";
            else st.pop();
        } else if(op == "query"){
            if(st.size() == 0) cout << "Anguei!\n";
            else cout << st.top() << '\n';
        } else {
            cout << st.size() << '\n';
        }
    }
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int t = 1;
    cin >> t;
    while(t--){
        solve();
    }
    return 0;
```

```
}
```

B - 队列

简单模拟即可

```
void solve() {
    int n;
    cin>>n;
    queue<int> q;
    while (n--) {
        int op;
        cin>>op;
        if (op==1) {
            int x;
            cin>>x;
            q.push(x);
        }
        else if (op==2) {
            if (q.size()) q.pop();
            else cout<<"ERR_CANNOT_POP"<<endl;
        }
        else if (op==3) {
            if (q.size()) cout<<q.front()<<endl;
            else cout<<"ERR_CANNOT_QUERY"<<endl;
        }
        else {
            cout<<q.size()<<endl;
        }
    }
}
```

C - 双端队列

本题如果使用**std:deque**会MLE，可以使用空间更小的**std:list**代替。

此外，手搓双向链表也是可以通过的。

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;
ll MAXN=1e6+5;
struct node{
    node *pre;
    node *next;
    int val;

    node():pre(0),next(0),val(0){}
};
vector<node*> bgn(MAXN);     //双向链表头结点
vector<node*> ed(MAXN);      //双向链表尾节点
vector<int> sz(MAXN);        //双向链表大小

void solve(){
    int t;
    cin>>t;

```

```
    for(int i=0;i<MAXN;i++){
        bgn[i]=new node();
        ed[i]=new node();
        bgn[i]->next=ed[i];
        ed[i]->pre=bgn[i];
    }

    while(t--){
        string com;
        cin>>com;
        int i;
        cin>>i;
        if(com=="pop_back") {
            if(sz[i]==0) continue;
            node* pop = ed[i]->pre;
            pop->pre->next=ed[i];
            ed[i]->pre=pop->pre;
            delete pop;
            sz[i]--;

        } else if(com=="pop_front") {
            if(sz[i]==0) continue;
            node* pop = bgn[i]->next;
            pop->next->pre=bgn[i];
            bgn[i]->next=pop->next;
            delete pop;
            sz[i]--;

        }else if(com=="push_back"){
            node *nd=new node();
            cin>>nd->val;
            nd->pre=ed[i]->pre;
            ed[i]->pre->next=nd;
            ed[i]->pre=nd;
            nd->next=ed[i];
            sz[i]++;

        }else if(com=="push_front"){
            node *nd=new node();
            cin>>nd->val;
            nd->next=bgn[i]->next;
            bgn[i]->next->pre=nd;
            bgn[i]->next=nd;
            nd->pre=bgn[i];
            sz[i]++;

        }else if(com=="size"){
            cout<<sz[i]<<endl;

        }else if(com=="back"){
            if(sz[i]==0)continue;

            cout<<(ed[i]->pre->val)<<endl;
        }else if(com=="front"){
            if(sz[i]==0)continue;
            cout<<(bgn[i]->next->val)<<endl;

        }
    }
}

```

```
//使用deque会被卡空间，可以使用std::list更简洁的双向链表，可以直接调库，不需要具体实现。

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t=1;
    //cin>>t;
    while(t--)solve();
    return 0;
}
```

D - 堆

优先队列 **priority_queue** 的板子题

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
using u64 = unsigned long long;
using i128 = __int128_t;


void solve(){
    priority_queue<int, vector<int>, greater<int>> q;
    int t, op, x;
    cin >> t;
    while(t--){
        cin >> op;
        if(op == 1){
            cin >> x;
            q.push(x);
        } else if(op == 2){
            cout << q.top() << '\n';
        } else q.pop();
    }
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--){
        solve();
    }
    return 0;
}
```

E_Seinfeld

给定一个由 `{` 和 `}` 组成的非空字符串，求最少需要多少次操作（将 `{` 变为 `}` 或反之）才能使字符串变得稳定（Stable）。
稳定的定义：

1. 空串是稳定的。
2. 如果 S 是稳定的，则 `{S}` 也是稳定的。

3. 如果 S, T 是稳定的，则 `ST` 也是稳定的。

思路

使用栈来进行括号匹配。

遍历字符串，如果遇到 `}` 且栈顶是 `{`，则说明找到了一对合法的 `{}`，将其消去（出栈）；否则将当前字符入栈。
遍历结束后，栈中剩余的字符既然无法消除，其形式必然是**若干个右括号紧跟若干个左括号**，即 `}}}...{{{`。

计算剩余贡献

设剩余的 `}` 数量为 `cnt2`，`{` 数量为 `cnt1`。

同类配对：

- 对于连续的两个 `{{`，我们只需要将其中一个改为 `}` 即可变为 `{}`，花费 1 次操作。
- 对于连续的两个 `}}`，我们只需要将其中一个改为 `{` 即可变为 `{}`，花费 1 次操作。

异类配对：

- 如果 `cnt1` 和 `cnt2` 都是奇数，处理完所有的同类配对后，会剩下一个 `}` 和一个 `{`，即 `}{`。要将 `}{` 变为 `{}`，需要将 `}` 变 `{`，且 `{` 变 `}`，共花费 2 次操作。

我们可以认为每个部分都需要改变一半的数量来完成匹配。即 `cnt1` 个 `{` 需要 $\lceil \frac{cnt1}{2} \rceil$ 次操作，`cnt2` 个 `}` 需要 $\lceil \frac{cnt2}{2} \rceil$ 次操作。

- 在整数运算中，$\lceil \frac{x}{2} \rceil$ 等价于 `(x + 1) / 2`。
- 验证一下 `}{` 的情况：`cnt1=1, cnt2=1`，公式计算为 `(1+1)/2 + (1+1)/2 = 1 + 1 = 2` 次，符合事实。
  因此最终答案为：

`(cnt1 + 1) / 2 + (cnt2 + 1) / 2`。

代码实现

```
void solve()
{
    stack<char> sta;
    int len = s.length();
    int com = 0;
    for (int i = 0; i < len; i++)
    {
        if (!sta.empty()&&sta.top() == '{' && s[i] == '}')
        {
            sta.pop();
            continue;
        }
        sta.push(s[i]);
    }
    int cnt1 = 0, cnt2 = 0;
    while (!sta.empty())
    {
        if (sta.top() == '{')
            cnt1++;
        else
            cnt2++;
        sta.pop();
    }

    com = (cnt1+1)/2+(cnt2+1)/2;
    cout << cnt << ". " << com << endl;
```

```
}
```

F - Train Problem I

对于给定的火车序列，可以分别用一个栈和队列来维护进入和退出的序列。当退出队列队首与进入队列栈顶相同时，说明此火车出栈时不会被其他火车阻挡，则栈和队列同时弹出，最后判断栈和队列中是否还有无法处理的元素即可判断是否合法。

```cpp
#include <bits/stdc++.h>
#define ll long long
using namespace std;

void solve(){
    int n;
    while(cin>>n){
        string in;
        string out;
        cin>>in;
        cin>>out;
        stack<char> stk;
        queue<char> que;
        queue<string> ans;

        for(int i=0;i<n;i++){
            que.push(out[i]);
        }
        for(int i=0;i<n;i++){
            stk.push(in[i]);
            ans.push("in");
            while(!stk.empty()&&!que.empty()
                  &&stk.top()==que.front()){
                ans.push("out");
                stk.pop();
                que.pop();
            }
        }
        if(!stk.empty()||!que.empty()){
            cout<<"No.\n";
        }else{
            cout<<"Yes.\n";
            while(!ans.empty()){
                cout<<ans.front()<<"\n";
                ans.pop();
            }
        }
        cout<<"FINISH\n";
    }
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t=1;
    //cin>>t;
    while(t--)solve();
    return 0;
}
```

G - 机器翻译

注意到 $1 \le M \le 100, 1 \le N \le 1000$，这个数据量就算是每一次从头到尾遍历一遍当前字典都没有问题，可以使用双端队列 **deque** ~~(因为头删除函数好写，还可以遍历)~~， 时间复杂度为 $O(MN)$

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
using u64 = unsigned long long;
using i128 = __int128_t;


void solve(){
    deque<int> q;
    int n, m, ans = 0, a;
    cin >> n >> m;
    while(m--){
        cin >> a;
        int flag = 0;
        for(int i = 0; i < q.size(); i++){
            if(q[i] == a){
                flag = 1;
                break;
            }
        }
        if(flag) continue;
        ans++;
        if(q.size() == n) q.pop_front();
        q.push_back(a);
    }
    cout << ans << '\n';
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--){
        solve();
    }
    return 0;
}
```

H_单调栈

模版题，单调栈维护即可

```
void solve() {
    int n;
    cin>>n;
    vector<int> a(n+5);
    for (int i=1;i<=n;i++) cin>>a[i];
    stack<int> sta;
```

```
    vector<int> posr(n+5);
    for(int i=n;i>=1;i--)
    {
        while(!sta.empty()&&a[sta.top()]<=a[i]) sta.pop();
        if(sta.empty()) posr[i]=0;
        else posr[i]=sta.top();
        sta.push(i);
    }
    for(int i=1;i<=n;i++)
    {
        cout<<posr[i]<<" ";
    }
    cout<<endl;
}
```

I - 单调队列 / 滑动窗口

将取区间最大值和区间最小值看成两个步骤，我们先关注区间最大值如何获得。

设想一个情景，若滑动窗口内已经存在若干个元素，当窗口向后滑动时，后一个元素如果比窗口内的所有元素都大，那么我们没有必要继续记录窗口内的其他元素，只记录新加入的元素即可（因为其他元素比这个元素小，他们在窗口内持续的时间也比这个新元素短）

由此可以推理至一般情况，对于进入窗口的元素，我们将其与窗口内的元素进行比较，并删除窗口内的比该元素小的元素，而对于正常退出窗口的元素，如果它还在我们记录的队列之中，则将其删除。

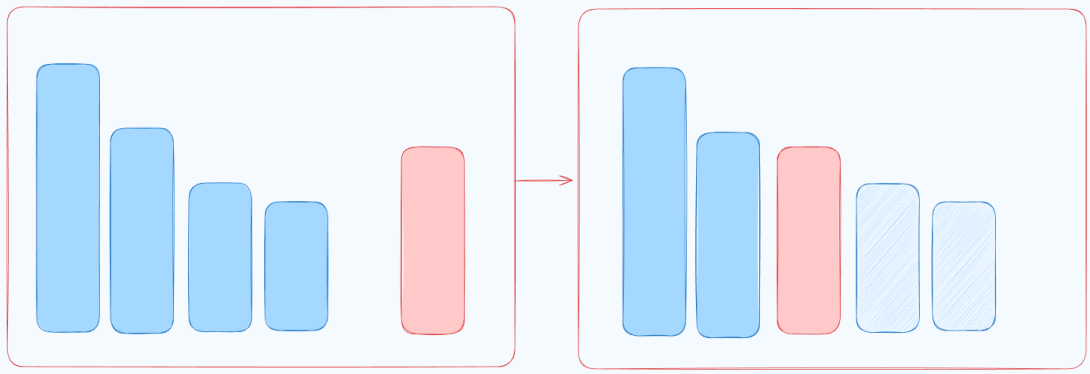
这个数据结构呈现一种不增的单调性，我们可以使用双端队列进行维护。

当窗口进行滑动时，由于队列内元素有序，对于新加入的新元素，我们一直比较新元素与队列尾端元素，若新元素较大则删除队尾元素，直到新元素小于等于队列中的所有元素（维护单调不增）。对于因为滑动而删除的旧元素，我们先判断这个元素是不是与此时队列队首元素（即最大的元素）相等，如果不等说明这个元素已经被之前的某步新元素入队操作淘汰掉了，所以不需要操作。如果相等说明没有被淘汰过，此时从队首弹出该元素。

这样操作，在每次移动后都输出队首元素，即为每个滑动窗口的最大元素。

最小元素的获取同理，可以同步操作（跑一遍滑动窗口同时维护两个deque）也可以异步操作（分别跑一遍最大窗口和最小窗口）。

时间复杂度$O(N)$。



因为红色元素值比前两个蓝色元素大，持续时间（寿命）也比前两个元素长
两个蓝色元素一定不会产生新的贡献，因此被淘汰，这样可以维护区间的有序性

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;

```

```
void solve(){
    int n,k;
    cin>>n>>k;
    vector<ll> vec(n);
    vector<ll> bg;
    vector<ll> sm;
    for(int i=0;i<n;i++){
        cin>>vec[i];
    }

    //单调不增单调队列
    deque<int> que;
    que.push_back(vec[0]);
    for(int i=0;i<k;i++){
        while(!que.empty()&&vec[i]>que.back()){
            que.pop_back();
        }
        que.push_back(vec[i]);
    }
    bg.push_back(que.front());
    for(int i=0;i+k<n;i++){
        int add=i+k;
        while(!que.empty()&&vec[add]>que.back()){
            que.pop_back();
        }
        que.push_back(vec[add]);

        if(vec[i]==que.front()){
            que.pop_front();
        }
        bg.push_back(que.front());
    }

    //单调不减单调队列
    que.clear();
    que.push_back(vec[0]);
    for(int i=0;i<k;i++){
        while(!que.empty()&&vec[i]<que.back()){
            que.pop_back();
        }
        que.push_back(vec[i]);
    }
    sm.push_back(que.front());
    for(int i=0;i+k<n;i++){
        int add=i+k;
        while(!que.empty()&&vec[add]<que.back()){
            que.pop_back();
        }
        que.push_back(vec[add]);

        if(vec[i]==que.front()){
            que.pop_front();
        }
        sm.push_back(que.front());
    }


    for(int i=0;i<sm.size();i++){
        cout<<sm[i]<<" ";
    }
    cout<<endl;
```

```
    for(int i=0;i<bg.size();i++){
        cout<<bg[i]<<" ";
    }
    cout<<endl;


}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t=1;
    //cin>>t;
    while(t--)solve();
    return 0;
}
```

J - 学籍管理

数据结构 **map** 的认识

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
using u64 = unsigned long long;
using i128 = __int128_t;


void solve(){
    map<string,int> mp;
    int n;
    cin >> n;
    while(n--){
        int op, val;
        string name;
        cin >> op;
        if(op == 1){
            cin >> name >> val;
            mp[name] = val;
            cout << "OK\n";
        } else if(op == 2){
            cin >> name;
            if(!mp.count(name)) cout << "Not found\n";
            else cout << mp[name] << '\n';
        } else if(op == 3){
            cin >> name;
            if(!mp.count(name)) cout << "Not found\n";
            else {
                mp.erase(name);
                cout << "Deleted successfully\n";
            }
        } else {
            cout << mp.size() << '\n';
        }
    }
}
```

```

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--){
        solve();
    }
    return 0;
}
```

K - Largest Rectangle in a Histogram

算法思想：单调栈。

核心逻辑：`最大矩形的高度一定等于某个柱子的高度`。对于每个柱子，向左右寻找第一个比它矮的柱子作为边界，利用单调栈可以在 O(N) 时间内计算出每个柱子的左右边界。

```
const int N=1e5+5;
int n;
int arr[N];
int ll[N];
int rr[N];
void solve()
{
    memset(arr,0,sizeof arr);
    memset(ll,0,sizeof ll);
    memset(rr,0, sizeof rr);
    for(int i=1;i<=n;i++)
    {
        cin>>arr[i];
    }
    stack<int> star;
    for(int i=n;i>0;i--)
    {
        while(!star.empty()&&arr[i]<=arr[star.top()])
        {
            star.pop();
        }
        if(star.empty())
        {
            rr[i]=n+1;
        }
        else
        {
            rr[i]=star.top();
        }
        star.push(i);
    }

    stack<int> stal;
    for(int i=1;i<=n;i++)
    {
        while(!stal.empty()&&arr[i]<=arr[stal.top()])
        {
            stal.pop();
        }
        if(stal.empty())
```

```
        {
            ll[i]=0;
        }
        else
        {
            ll[i]=stal.top();
        }
        stal.push(i);
    }

    int maxn=0;
    for(int i=1;i<=n;i++)
    {
        maxn=max(maxn,(rr[i]-ll[i]-1)*arr[i]);
    }
    cout<<maxn<<endl;

}
```

L - 海港

使用队列维护每条船的时间和船上的人的种类，同时维护当前总种类数**ans**，使用**mem**维护每个种类的人出现的次数，通过判断时间来进行船的入队出队操作。

人数增加且此时`mem[i]==0`时`ans++`

人数减少且此时`mem[i]==1`时`ans--`

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;
ll MAXN=1e5+5;
struct node{
    ll t;
    vector<int> mem;
};

void solve(){
    int n;
    cin>>n;
    vector<int> mem(MAXN);
    int ans=0;
    queue<node> que;
    for(int i=0;i<n;i++){
        node cur;
        cin>>cur.t;
        int k;
        cin>>k;
        for(int j=0;j<k;j++){
            int in;
            cin>>in;
            cur.mem.push_back(in);
            if(mem[in]==0)ans++;
            mem[in]++;
        }

        que.push(cur);
        while(que.front().t<=cur.t-86400){
            for(int i:que.front().mem){
                if(mem[i]==1)ans--;
```

```
                mem[i]--;
            }
            que.pop();
        }

        cout<<ans<<"\n";
    }
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t=1;
    //cin>>t;
    while(t--)solve();
    return 0;
}
```

M - Set To Max (Hard version)

题目的大意为， 通过用 **区间最大值** 来更新这段区间，试图让两个数组相等。我们可以分类讨论：

1. 当 $a_i = b_i$ 时， 直接跳过无需考虑；
2. 当 $a_i > b_i$ 时，显然 a_i 无法变小，直接输出 NO 即可；
3. 当 $a_i < b_i$ 时，我们需要考虑用 a_i 附近的 b_i 来试图更新：

   贪心思考，如果 a_i 右侧有 a_j 、a_k $(j < k)$ 同时等于 b_i，显然用 a_k 更新会影响到更多其他的数组值的同时，对于 a_i 的更新并没有帮助，所以我们只需要考虑 a_i **左侧与右侧最近** 的 $a_j = b_i$ 即可

   这样的更新需要满足两个条件：

   ① 对于 a 数组来说，比如 $a_i, a_{i+1} \dots a_r (a_r = b_i)$， 那么我需要保证这些数值不会有 $a_j > b_i$ 发生，否则我就无法用 a_r 来合法地更新 a_i ，我就需要输出 NO， 即

   $$max\{a_i, a_{i+1} \dots a_r\} \le b_i$$

   ② 对于b数组来说，我不能自私地只管让 a_i 变成 b_i 就行了，如果在这个过程中我让其他数字 $a_j > b_j$ ，这就一定是一个失败的非法操作，所以要保证

   $$min\{b_i, b_{i+1} \dots b_r\} \ge b_i$$

   保证这两个条件，我们对每一位单独的 a_i 处理的时候就不会影响到周边其他的数字,就可以对 a 数组进行遍历检查每一位是否合法，现在需要考虑优化时间复杂度，所有操作暴力复杂度为 $O(n^2)$，其中遍历操作很难被优化不考虑，对于迅速找出距离 a_i 最近的 b_i，我们可以开一个二维数组 **vector** 记录每一个 a_i 值出现的位置，用lower_bound函数(或者自己写二分)快速找到第一个大于索引 i 的位置即为右侧位置，他的上一个位置prev即为左侧位置，当然要保证他们是存在非空的，否则输出NO；对于区间最值用 **st表** 、**树状数组** 、 **线段树** 之类的数据结构均可以，此处采用st表, 时间复杂度 $O(nlogn)$

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
using u64 = unsigned long long;
using i128 = __int128_t;


void solve(){
    int n;
    cin >> n;
```

```
    vector<int> a(n), b(n);
    vector<vector<int>> pos(n + 1);
    for(int i = 0; i < n; i++){
        cin >> a[i];
        pos[a[i]].push_back(i);
    }
    for(int i = 0; i < n; i++) cin >> b[i];
    //建立st表
    vector<int> lg(n + 1);
    for(int i = 2; i <= n; i++){
        lg[i] = lg[i / 2] + 1;
    }
    vector<vector<int>> fa(n + 1, vector<int> (40)), fb(n + 1, vector<int> (40));
    for(int i = 0; i < n; i++){
        fa[i][0] = a[i];
        fb[i][0] = b[i];
    }
    for(int j = 1; j <= lg[n] + 1; j++){
        for(int i = 0; i + (1 << j) <= n; i++){
            fa[i][j] = max(fa[i][j - 1], fa[i + (1 << j - 1)][j - 1]);
            fb[i][j] = min(fb[i][j - 1], fb[i + (1 << j - 1)][j - 1]);
        }
    }

    for(int i = 0; i < n; i++){
        if(a[i] == b[i]) continue;
        if(a[i] > b[i]){
            cout << "NO\n";
            return;
        }
        auto it = lower_bound(pos[b[i]].begin(), pos[b[i]].end(), i);
        if(pos[b[i]].empty()){
            cout << "NO\n";
            return;
        }

        int flag = 0;

        if(it != pos[b[i]].end()){
            int idx = *it;
            int s = lg[idx - i + 1];
            if(max(fa[i][s], fa[idx - (1 << s) + 1][s]) <= b[i] && min(fb[i][s], fb[idx - (1 << s) + 1][s]) >= b[i]){
                flag = 1;
            }
        }

        if(it != pos[b[i]].begin()){
            int idx = *prev(it);
            int s = lg[i - idx + 1];
            if(max(fa[idx][s], fa[i - (1 << s) + 1][s]) <= b[i] && min(fb[idx][s], fb[i - (1 << s) + 1][s]) >= b[i]){
                flag = 1;
            }
        }

        if(!flag){
            cout << "NO\n";
            return;
        }
    }
```

```

    cout << "YES\n";
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int t = 1;
    cin >> t;
    while(t--){
        solve();
    }
    return 0;
}
```

N - Running Median

算法思想：对顶堆。

核心逻辑：使用`最大堆`维护较小的一半元素，`最小堆`维护较大的一半元素。通过动态调整两个堆的元素，使得中位数始终处于`最大堆`的堆顶。

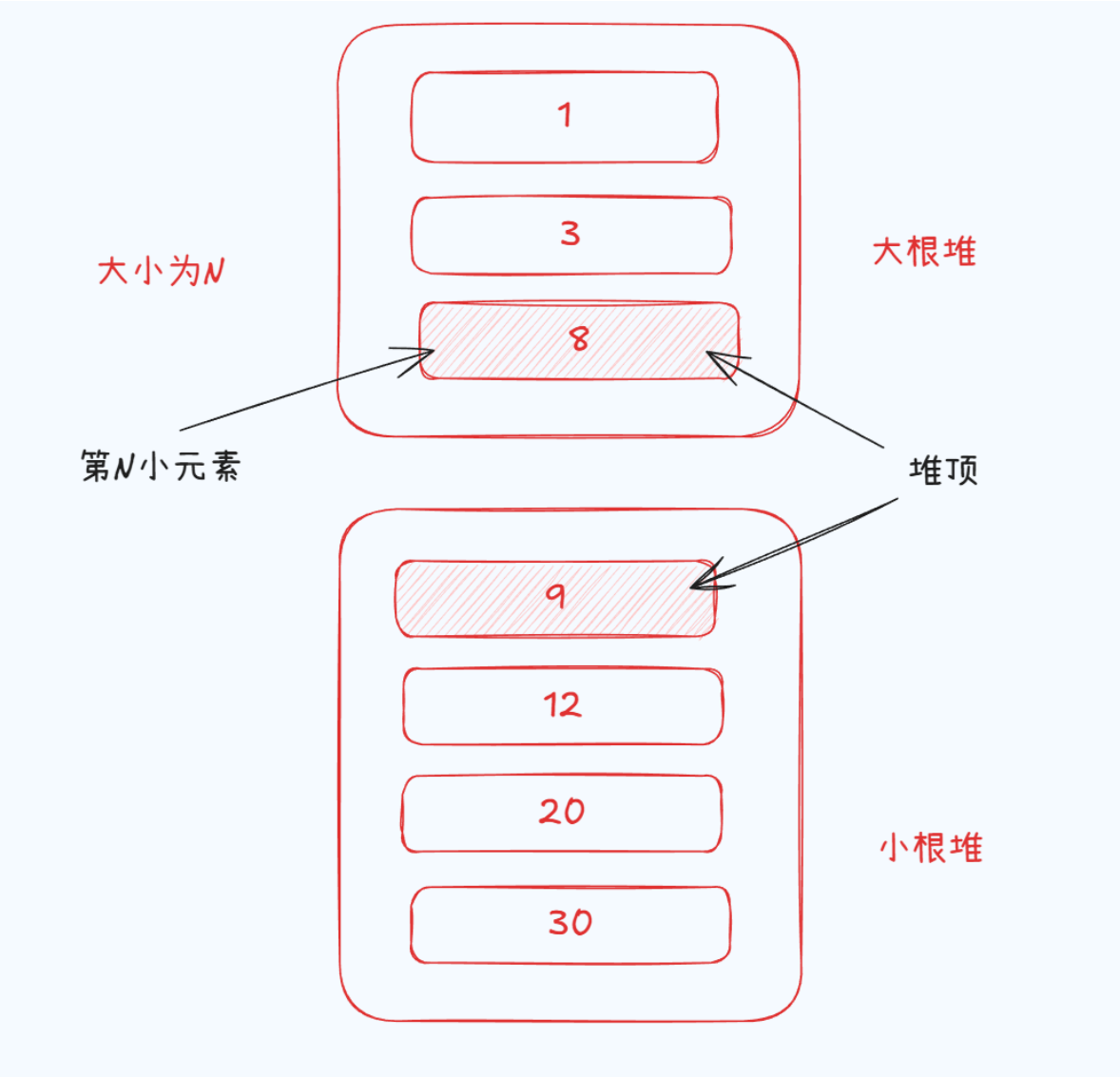
```
const int N=1e4+5;
int q,n,id;
int arr[N];

void solve()
{
    cin>>id>>n;
    for(int i=1;i<=n;i++)
    {
        cin>>arr[i];
    }
    priority_queue<int> q1;//队头维护中位数
    priority_queue<int,vector<int>,greater<int> > q2;
    cout<<id<<' '<<n/2+1<<endl;
    vector<int> mid;

    for(int i=1;i<=n;i++)
    {
        q1.push(arr[i]);
        if(i==1)
        {
            mid.push_back(arr[i]);
        }
        if(i%2==1&&i>1)
        {
            int x=q1.top();
            q1.pop();
            q2.push(x);
            mid.push_back(q1.top());
        }
        while(!q1.empty()&&!q2.empty()&&q1.top()>q2.top())
        {
            int x=q1.top(),y=q2.top();
            q1.pop(),q2.pop();
            q1.push(y),q2.push(x);
        }
    }
```

```
    int cnt=1;
    for(int x:mid)
    {
        cout<<x<<' ';
        cnt++;
        if(cnt%10==0)
        cout<<endl;
    }
    cout<<endl;
}
```

O - 黑匣子

原理同N题，同样维护两个堆，一个大根堆一个小根堆，大根堆中所有元素全部小于小根堆，其中大小为N的大根堆的堆顶元素即为所求的第N小元素。

通过维护两个堆顶元素大小比较和互相插入弹出即可维护大根堆所有元素比小根堆都小的性质。




```
#include <bits/stdc++.h>
#define ll long long
using namespace std;

void solve(){
    int n,m;
    cin>>n>>m;
```

```
    vector<ll> vec(n+1);
    for(int i=1;i<=n;i++){
        cin>>vec[i];
    }

    queue<ll> add;
    for(int i=0;i<m;i++){
        ll in;
        cin>>in;
        add.push(in);
    }

    priority_queue<ll,vector<ll>,greater<ll>> smlque;
    priority_queue<ll> bigque;
    ll sz=0;
    for(int i=1;i<=n;i++){
        if(bigque.empty()||vec[i]<=bigque.top()){
            bigque.push(vec[i]);
        }else{
            smlque.push(vec[i]);
        }
        while(i==add.front()){
            sz++;
            while(bigque.size()>sz){
                smlque.push(bigque.top());
                bigque.pop();
            }
            while(bigque.size()<sz){
                bigque.push(smlque.top());
                smlque.pop();
            }
            cout<<bigque.top()<<"\n";
            add.pop();
            if(add.empty())return;
        }
    }

}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t=1;
    //cin>>t;
    while(t--)solve();
    return 0;
}
```

P - 简单计算器

模拟，保证乘除法优先级高于加减法。可以直接模拟，单栈或双栈。
唯一要额外注意的是这道题杭电OJ卡精度，比如13.355在计算机中可能被存储为13.3549999999...，导致四舍五入会为13.35而不是13.36，这里可以用 `round(ans * 100) / 100` 强制四舍五入， 也可以对正数(负数)加上(减去)一个极小的数解决

```
#include <bits/stdc++.h>
using namespace std;
```

```


int get(char op){
    if(op == '-' || op == '+') return 1;
    if(op == '*' || op == '/') return 2;
    return 0;
}

double cal(double a,double b, char op){
    if(op == '-') return a - b;
    if(op == '+') return a + b;
    if(op == '*') return a * b;
    if(op == '/') return a / b;
    return 0;
}

void solve() {
    string s;
    while(getline(cin, s)){
        if(s == "0") break;
        stack<char> sc;
        stack<double> st;
        for(int i = 0; i < s.size(); i++){
            if(s[i] == ' ') continue;
            else if(s[i] >= '0' && s[i] <= '9'){
                double a = 0.0;
                while(i < s.size() && s[i] >= '0' && s[i] <= '9'){
                    a *= 10;
                    a += s[i] - '0';
                    i++;
                }
                st.push(a);
                i--;
            } else if(s[i] == '+' || s[i] == '-' || s[i] == '*' || s[i] == '/'){
                while(!sc.empty() && get(sc.top()) >= get(s[i])){
                    double b = st.top(); st.pop();
                    double a = st.top(); st.pop();
                    char op = sc.top(); sc.pop();
                    st.push(cal(a,b,op));
                }
                sc.push(s[i]);
            }

        }
        while(!sc.empty()){
            double b = st.top(); st.pop();
            double a = st.top(); st.pop();
            char op = sc.top(); sc.pop();
            st.push(cal(a, b, op));
        }
        double ans = st.top();
        if (ans > 0) ans += 1e-8;
        else ans -= 1e-8;
        cout << fixed << setprecision(2) << ans << '\n';
    }


}

int main() {
    ios::sync_with_stdio(false);
```

```
    cin.tie(0), cout.tie(0);

    // int t;
    // cin >> t;
    // while (t--)
    solve();

    return 0;
}
```

Q - ST 表 & RMQ 问题

`使用'\n'替代endl换行，不然会被卡时间`

算法思想：ST 表（Sparse Table）

核心逻辑：基于倍增思想和动态规划。

1. **预处理 (Build)**: `st[i][j]` 表示从下标 i 开始，长度为 2^j 的区间内的最大值。状态转移方程：`st[i][j] = max(st[i][j-1], st[i + (1<<(j-1))][j-1])`。时间复杂度 $O(N \log N)$。
2. **查询 (Query)**: 对于查询区间 $[l, r]$，找到一个 k 使得 $2^k \le r - l + 1$。最大值即为 `max(st[l][k], st[r - (1<<k) + 1][k])`。由于两个长度为 2^k 的区间覆盖了整个 $[l, r]$（允许重叠），故可在 $O(1)$ 时间内得到答案。

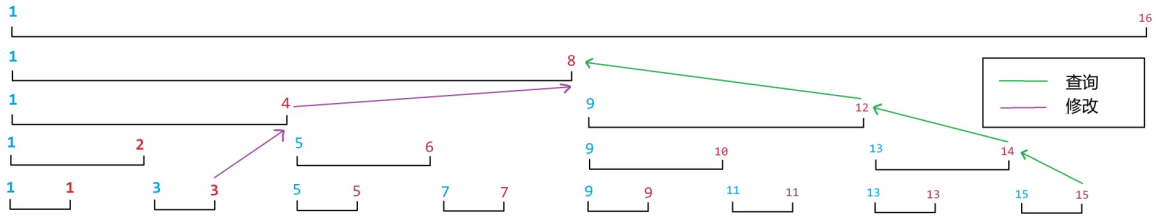
```
const int N=1e5+5;
const int M=30;
int n,m;
int st[N][M];
int a[N];

inline int read()
{
    int x=0,f=1;char ch=getchar();
    while (ch<'0'||ch>'9'){if (ch=='-') f=-1;ch=getchar();}
    while (ch>='0'&&ch<='9'){x=x*10+ch-48;ch=getchar();}
    return x*f;
}

void st_build()
{
    for(int i=1;i<=n;i++) st[i][0]=a[i];
    int k=log2(n);
    for(int j=1;j<=k;j++)
    {
        for(int i=1;i+(1<<j)-1<=n;i++)
        {
            st[i][j]=max(st[i][j-1],st[i+(1<<(j-1))][j-1]);
        }
    }
}

int st_query(int l,int r)
{
    int k=log2(r-l+1);
    return max(st[l][k],st[r-(1<<k)+1][k]);
}

void solve()
{
    n=read();m=read();
```

```
    for(int i=1;i<=n;i++) a[i]=read();
    st_build();
    while(m--)
    {
        int l,r;
        l=read();r=read();
        cout<<st_query(l,r)<<endl;
    }
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    int q=1;
    //cin>>q;
    while(q--)
    {
        solve();
    }
    return 0;
}
```

R - 树状数组 1

树状数组（BIT）是一种便捷处理区间操作的数据结构，他的基本思想是使用二分的思想将一个区间划分为若干个子区间，并使用一部分元素维护子区间的和或其他关系。

其中使用`lowbit`来快速选取下一个应访问元素。



对于本题，则是使用树状数组进行单点修改，区间查询区间和。

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;

ll MAXN = 5e5+5;
vector<int> vec(MAXN);

int lowbit(int a){
    return a&(-a);
}

int sum(int a){
    int res=0;
    for(int i=a;i>0;i-=lowbit(i)){
        res+=vec[i];
    }
    return res;
}

```

```
int n,m;
void add(int a,ll b){
    for(int i=a;i<=n;i+=lowbit(i)){
        vec[i]+=b;
    }
}


void solve(){
    cin>>n>>m;
    vector<int> ori(n+1);
    for(int i=1;i<=n;i++){
        cin>>ori[i];
        add(i,ori[i]);
    }

    for(int i=0;i<m;i++){
        int a,b,c;
        cin>>a>>b>>c;
        if(a==1){
            add(b,c);
        }else{
            cout<<sum(c)-sum(b-1)<<"\n";
        }
    }
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t=1;
    //cin>>t;
    while(t--)solve();
    return 0;
}
```

S - 树状数组 2

树状数组1的要求为单点修改，区间查询(求和)，我们可以运用 **差分** 的思想，把树状数组里面的 $c_i = a_i - a_{i-1}$，这样 $\sum_{i=0}^{n} c_i = a_i (a_0 = 0)$，可以起到单点查询的功能，对于区间修改，如果是对 $[l, r]$ 区间加，体现在差分上为 $c_l + k, c_{r+1} - k$

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;
using u64 = unsigned long long;
using i128 = __int128_t;

struct BIT{
    int n;
    vector<i64> c;
    BIT(int n, vector<int>& a) : n(n){
        c.resize(n + 1, 0);
        for(int i = 1; i <= n; i++){
            add(i, a[i] - a[i - 1]);
        }
```

```cpp
    }
    int lowbit(int x){return x & -x;}
    i64 sum(int x){
        i64 res = 0;
        while(x){
            res += c[x];
            x -= lowbit(x);
        }
        return res;
    }
    void add(int x, int k){
        while(x <= n){
            c[x] += k;
            x += lowbit(x);
        }
    }
};

void solve(){
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1);
    for(int i = 1; i <= n; i++){
        cin >> a[i];
    }
    BIT bit(n, a);
    while(m--){
        int op, x, y, k;
        cin >> op;
        if(op == 1){
            cin >> x >> y >> k;
            bit.add(x, k);
            bit.add(y + 1, -k);
        } else {
            cin >> x;
            cout << bit.sum(x) << '\n';
        }
    }
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int t = 1;
    //cin >> t;
    while(t--){
        solve();
    }
    return 0;
}
```

T - 逆序对

算法思想：归并排序 (Merge Sort) / 分治。

核心逻辑：在归并排序的合并过程中计算逆序对。

1. **分解**: 将区间 $[l, r]$ 平分为 $[l, mid]$ 和 $[mid+1, r]$，递归处理。
2. **合并与统计**:

- 设左半部分指针 i，右半部分指针 j。
- 当 $q[i] \le q[j]$ 时，正常归并。
- 当 $q[i] > q[j]$ 时，说明 $q[i]$ 及其后面所有在左半部分剩余的元素（即 $mid - i + 1$ 个）都比 $q[j]$ 大，且它们在 $q[j]$ 左侧，因此构成了 $mid - i + 1$ 对逆序对。累加答案即可。
- 通过分治将问题规模缩小，时间复杂度为 $O(N \log N)$。

```cpp
#include <iostream>
using namespace std;

typedef long long ll;
const int N = 5e5+10;
int n;
int q[N];
int tmp[N];

ll merge_sort(int q[],int l, int r)
{
    if (l >= r) return 0;
    int mid = l + r >> 1;
    ll res = merge_sort(q,l, mid) + merge_sort(q,mid + 1, r);
    //归并
    int k = 0, i = l, j = mid + 1;
    while (i <= mid && j <= r)
    {
        if (q[i] <= q[j])
            tmp[k++] = q[i++];
        else
        {
            tmp[k++] = q[j++];
            res+=mid-i+1;
        }
    }
    while (i <= mid)
        tmp[k++] = q[i++];
    while (j <= r)
        tmp[k++] = q[j++];
    for (int i = l, j = 0; i <= r; i++, j++)
    {
        q[i] = tmp[j];
    }
    return res;
}

int main()
{
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cin >> q[i];
    }
    cout << merge_sort(q,0, n - 1) << endl;
}
```

U - 上帝造题的七分钟

二维树状数组进行区间修改区间查询

区间修改需要使用差分数组，区间查询需要对原数组求前缀和，即对差分数组的前缀和再求前缀和

设差分数组$d[i][j]$，原数组$a[i][j]$，前缀和数组$sum[i][j]$

对于二维差分，$d[i][j]=a[i][j]-a[i][j-1]-a[i-1][j]+a[i-1][j-1]$

对于区间查询，二维前缀和的公式为：

$sum[x][y]=\sum_{i=1}^{x}\sum_{j=1}^{y}a[i][j]=\sum_{i=1}^{x}\sum_{j=1}^{y}d[i][j]\times(x-i+1)\times(y-j+1)$

$=\sum_{i=1}^{x}\sum_{j=1}^{y}d[i][j]\times(xy+x+y+1)-d[i][j]\times i(y+1)-d[i][j]\times j(x+1)+d[i][j]\times(i\times j)$

即需要维护 `d[i][j]`，`d[i][j]*i`，`d[i][j]*j`，`d[i][j]*i*j` 四个树状数组

对于区间修改，即同时修改$d[i][j]$，$d[i][j]*i$，$d[i][j]*j$，$d[i][j]*i*j$，记得给每一个加上的val乘上对应的i或j

```
#include <bits/stdc++.h>
#define ll long long
using namespace std;

ll MAXN = 2050;
int n,m;

struct bit{
    vector<vector<int>> vec;

    int lowbit(int a){
        return a&(-a);
    }

    int sum(int a,int b){
        int res=0;
        for(int i=a;i>0;i-=lowbit(i)){
            for(int j=b;j>0;j-=lowbit(j)){
                res+=vec[i][j];
            }
        }
        return res;
    }

    void add(int x,int y,ll b){
        for(int i=x;i<=n;i+=lowbit(i)){
            for(int j=y;j<=m;j+=lowbit(j)){
                vec[i][j]+=b;
            }
        }
    }

    bit():vec(MAXN,vector<int>(MAXN)){}
}A,A_i,A_j,A_ij;

int sum(int x,int y)
{
    return A.sum(x,y)*(x*y+x+y+1)-A_i.sum(x,y)*(y+1)-A_j.sum(x,y)*(x+1)+A_ij.sum(x,y);
}

void add(int x,int y,int num)
{
    A.add(x,y,num);
    A_i.add(x,y,num*x);
    A_j.add(x,y,num*y);
    A_ij.add(x,y,num*x*y);
```

```
}

void solve(){
    char x;
    cin>>x>>n>>m;
    vector<int> ori(n+1);

    while(cin>>x){
        if(x=='L'){
            int ax,ay,bx,by,val;
            cin>>ax>>ay>>bx>>by>>val;

            add(ax,ay,val);
            add(ax,by+1,-val);
            add(bx+1,ay,-val);
            add(bx+1,by+1,val);
        }else{
            int ax,ay,bx,by;
            cin>>ax>>ay>>bx>>by;
            cout<<sum(ax-1,ay-1)-sum(ax-1,by)-sum(bx,ay-1)+sum(bx,by)<<"\n";
        }
    }
}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t=1;
    //cin>>t;
    while(t--)solve();
    return 0;
}
```